Decentralized Proximal Gradient Algorithms With Linear Convergence Rates

Sulaiman A. Alghunaim, Ernest K. Ryu, Kun Yuan, and Ali H. Sayed, *Fellow, IEEE*

***Abstract*—This article studies a class of nonsmooth decentralized multiagent optimization problems where the agents aim at minimizing a sum of local strongly-convex smooth components plus a common nonsmooth term. We propose a general primal-dual algorithmic framework that unifies many existing state-of-the-art algorithms. We establish linear convergence of the proposed method to the exact minimizer in the presence of the nonsmooth term. Moreover, for the more general class of problems with agent specific nonsmooth terms, we show that linear convergence cannot be achieved (in the worst case) for the class of algorithms that uses the gradients and the proximal mappings of the smooth and nonsmooth parts, respectively. We further provide a numerical counterexample that shows how some state-of-the-art algorithms fail to converge linearly for strongly convex objectives and different local non smooth terms.**

***Index Terms*—Decentralized optimization, diffusion, gradient tracking, linear convergence, proximal gradient algorithms, unified decentralized algorithm.**

I. Introduction

In this article, we consider a static and undirected network of $K \geq 2$ agents connected over some graph where each agent k owns a private cost function $J_k : \mathbb{R}^M \to \mathbb{R}$. Through only local interactions (i.e., with agents only communicating with their immediate neighbors), each agent is interested in finding a solution to the following problem:

$$w^\star \in \arg\min_{w\in\mathbb{R}^M} \quad \frac{1}{K}\sum_{k=1}^{K} J_k(w) + R(w) \tag{1}$$

where $R : \mathbb{R}^M \to \mathbb{R} \cup \{+\infty\}$ is a convex function (not necessarily differentiable). We adopt the following assumption throughout this article.

Assumption 1 (Cost Function): *We assume that a solution exists to problem (1) and each cost function $J_k(w)$ is first-order differentiable and ν-strongly convex*

$$(w^o - w^\bullet)^{\mathsf{T}} \left(\nabla J_k(w^o) - \nabla J_k(w^\bullet)\right) \geq \nu \|w^o - w^\bullet\|^2 \tag{2}$$

with δ-Lipschitz continuous gradients

$$\|\nabla J_k(w^o) - \nabla J_k(w^\bullet)\| \leq \delta \|w^o - w^\bullet\| \tag{3}$$

for any w^o and $w^\bullet$. The constants ν and δ are strictly positive and satisfy $0 < \nu \leq \delta$. We also assume $R(w)$ to be a proper[1] and lower-semicontinuous convex function. □

Note that from the strong-convexity condition (2), we know that the objective function in (1) is also strongly convex and, thus, the global solution $w^\star$ is unique.

Related Works: Various algorithms have been proposed to solve decentralized optimization problems of the form (1) (see [1]–[14]). Only few works have attempted to unify some of these various algorithms [15]–[17]. For example, the work [15] proposed a general method that includes EXTRA [8] and DIGing [7] (for static and undirected network) as special cases. However, the method in [15] does not include the adapt-then-combine[2] (ATC) gradient-tracking algorithms [1], [2], [4]. The work [16] proposed a canonical form that characterizes decentralized algorithms that require a single round of communication and gradient computation per iteration, which does not include the Aug-DGM (ATC-DIGing) [1], [2]. The authors in [16] only focused on the canonical form without focusing on the analysis of this form. Later, the work [17] studied a class of the canonical form in [16] over time-varying connected networks and provided worst case linear convergence rates through numerical solution of semidefinite programs.

Different from [15]–[17] we propose a general primal-dual framework that unifies many existing gradient algorithms including EXTRA [8], DLM [9], exact diffusion [10], NIDS [11], and different implementations of the gradient tracking methods [1]–[4], [6] including Aug-DGM [1]. Our framework shows that the ATC gradient-tracking methods can be represented as primal-dual recursions. The work [20] proposed a proximal gradient algorithm that solves (1) and established its linear convergence to the *exact* solution $w^\star$. Motivated by the technique from [20] we extend the proposed general framework to handle the nonsmooth term $R(w)$ and prove linear convergence of the proposed general method to the solution $w^\star$ in the presence of the nonsmooth term.

In order to establish exact global linear convergence, this article considers the nonsmooth term to be common across all agents. One might wonder whether it is possible for a decentralized proximal gradient algorithm to achieve global linear convergence in the presence of different local nonsmooth $R_k(w)$ terms. As far as we know, this question has not been explicitly answered in the literature. Many

Manuscript received September 13, 2019; revised February 29, 2020; accepted July 3, 2020. Date of publication July 15, 2020; date of current version May 27, 2021. Recommended by Associate Editor Prof. J. Lavaei. *(Corresponding author: Sulaiman A. Alghunaim.)*

Sulaiman A. Alghunaim is with the Department of Electrical Engineering, Kuwait University, 5969, Safat, Kuwait (e-mail: salghunaim@ucla.edu).

Ernest K. Ryu is with the Department of Mathematical Sciences, Seoul National University, Seoul, 08826, South Korea (e-mail: ernestryu@snu.ac.kr).

Kun Yuan is with the Alibaba Group (US), Bellevue, WA 98004 USA (e-mail: kunyuan@ucla.edu).

Ali H. Sayed is with the School of Engineering, Ecole Polytechnique Federale de Lausanne (EPFL), 1015 Lausanne, Switzerland (e-mail: ali.sayed@epfl.ch).

Color versions of one or more of the figures in this article are available online at https://ieeexplore.ieee.org.

Digital Object Identifier 10.1109/TAC.2020.3009363

[1]Function $f(.)$ is proper if $-\infty < f(x)$ for all x in its domain and $f(x) < \infty$ for at least one x.

[2]Adapt-then-combine (ATC) structure was proposed in [18] to distinguish between different implementations of diffusion learning strategies (see also [19, Ch. 7]).

decentralized optimization problems where each agent k has a local nonsmooth term $R_k(w)$ possibly different from other agents have been proposed [11], [21]–[24]. None of these methods have been shown to achieve global linear convergence in the presence of general nonsmooth terms. By adjusting the results from [25] to the decentralized optimization setup with agent-specific nonsmooth terms $\{R_k(w)\}$, it can be shown that it is *impossible* for any proximal gradient-based algorithm to achieve linear convergence in the *worst case* (see Section VI). Note that the works [26], [27] showed that global linear convergence is not possible for nonsmooth strongly convex functions in the worst case for the class of algorithms limited to one communication round but unlimited in the amount of computation and access to the functions per iteration. In contrast, we consider algorithms unlimited in the number of communications rounds but limited to one gradient and proximal computations per iteration. We remark that under a common nonsmooth term, the work [14] also established global linear convergence for a decentralized algorithm that is based on successive convex approximation, which is different from our proximal primal-dual approach.

Contribution: Given the above, this article has three contributions. First, when $R(w) = 0$ we propose a novel primal-dual *unified decentralized algorithm* (UDA) that unifies many existing state-of-the-art algorithms including the ATC algorithms [1]–[4], [10], [11] and the non-ATC algorithms [6]–[9]. To our knowledge, this is the first primal-dual interpretation of the ATC gradient-tracking methods [1]–[4]. Second, we extend this framework to handle a *common* nonsmooth regularization term and provide a unifying linear convergence analysis under proper conditions. Our step size and convergence rate upper bounds shed light on the stability and performance of these various methods. Third, by tailoring a result from [25], we show that if each agent owns a nonsmooth term, then linear convergence to the *exact* solution $w^\star$ cannot be achieved in the worst case for the class of decentralized algorithms where each agent can compute one gradient and one proximal mapping per iteration for the smooth and nonsmooth parts, respectively. We further provide a numerical counterexample where PG-EXTRA [21] and proximal linearized ADMM [22], [23] fail to achieve global linear convergence for strongly convex objectives.

Notation: For a vector $x \in \mathbb{R}^M$ and a positive semidefinite matrix $C \geq 0$, we let $\|x\|_C^2 = x^\mathsf{T} C x$. For any matrix A, we let $\sigma_{\max}(A)$ denote the maximum singular value of A and $\underline{\sigma}(A)$ denote the minimum *nonzero* singular value of A. Moreover, for any symmetric matrices A and B with the same dimension, we let $A \geq B$ ($A > B$) if $A - B$ is positive semidefinite (positive definite). The identity matrix with size N is denoted by I_N. We let $\mathbb{1}_N$ be a vector of size N with all entries equal to one. The Kronecker product is denoted by $\otimes$. We let $\mathrm{col}\{x_n\}_{n=1}^N$ denote a column vector (matrix) that stacks the vector (matrices) x_n of appropriate dimensions on top of each other. The proximal operator with parameter $\mu > 0$ of a function $f : \mathbb{R}^M \to \mathbb{R}$ is $\mathbf{prox}_{\mu f}(x) = \arg\min_z f(z) + \frac{1}{2\mu}\|z - x\|^2$.

II. Unified Decentralized Algorithm (UDA)

In this section, we present the *unified decentralized algorithm* (UDA) that covers various state-of-the-art algorithms as special cases. To this end, we will first focus on the smooth case ($R(w) = 0$), which will then be extended to handle the nonsmooth component $R(w)$ in the following section.

A. General Primal-Dual Framework

For algorithm derivation and motivation purposes, we will rewrite problem (1) in an equivalent manner. To do that, we let $w_k \in \mathbb{R}^M$ denote a local copy of w available at agent k and introduce the network quantities

$$w \triangleq \mathrm{col}\{w_k\}_{k=1}^K \in \mathbb{R}^{KM},\ \mathcal{J}(w) \triangleq \frac{1}{K}\sum_{k=1}^K J_k(w_k). \tag{4}$$

Further, we introduce two general symmetric matrices $\mathcal{B} \in \mathbb{R}^{MK\times MK}$ and $\mathcal{C} \in \mathbb{R}^{MK\times MK}$ that satisfy the following conditions

$$\begin{cases} \mathcal{B}w = 0 \iff w_1 = \cdots = w_K & \text{(5a)} \\ \mathcal{C} = 0 \ \text{ or } \ \mathcal{C}w = 0 \iff \mathcal{B}w = 0. & \text{(5b)} \end{cases}$$

For algorithm derivation, the matrices $\{\mathcal{B}, \mathcal{C}\}$ can be any general consensus matrices [28]. Later, we will see how to choose these matrices to recover different decentralized implementations (see Section II-C). With these quantities, it is easy to see that problem (1) with $R(w) = 0$ is equivalent to the following problem:

$$\underset{\mathcal{W}\in\mathbb{R}^{KM}}{\text{minimize}} \quad \mathcal{J}(w) + \frac{1}{2\mu}\|w\|_{\mathcal{C}}^2, \quad \text{s.t. } \mathcal{B}w = 0 \tag{6}$$

where $\mu > 0$ and the matrix $\mathcal{C} \in \mathbb{R}^{MK\times MK}$ is a positive semidefinite consensus penalty matrix satisfying (5b). To solve problem (6), we consider the saddle-point formulation

$$\min_{\mathcal{W}} \max_{\mathcal{Y}} \quad \mathcal{L}(w, y) \triangleq \mathcal{J}(w) + \frac{1}{\mu} y^\mathsf{T}\mathcal{B}w + \frac{1}{2\mu}\|w\|_{\mathcal{C}}^2 \tag{7}$$

where $y \in \mathbb{R}^{MK}$ is the dual variable. To solve (7), we propose the following algorithm: Let $y_{-1} = 0$ and w_{-1} take any arbitrary value. Repeat for $i = 0, 1, \cdots$

$$\begin{cases} z_i = (I - \mathcal{C})w_{i-1} - \mu\nabla\mathcal{J}(w_{i-1}) - \mathcal{B}y_{i-1} & \text{(8a)} \\ y_i = y_{i-1} + \mathcal{B}z_i & \text{(8b)} \\ w_i = \bar{\mathcal{A}}z_i \qquad (\textbf{Combine}) & \text{(8c)} \end{cases}$$

where $\bar{\mathcal{A}} = \bar{A} \otimes I_M$ and $\bar{A}$ is a symmetric and doubly stochastic combination matrix. In the above UDA algorithm, step (8a) is a gradient descent followed by a gradient ascent step in (8b), both applied to the saddle-point problem (7) with step size μ. The last step (8c) is a combination step that enforces further agreement. Next we show that by proper choices of $\bar{\mathcal{A}}$, $\mathcal{B}$, and $\mathcal{C}$ we can recover many state-of-the-art algorithms. To do that, we need to introduce the combination matrix associated with the network.

B. Network Combination Matrix

Thus, we introduce the combination matrices

$$A = [a_{sk}] \in \mathbb{R}^{K\times K}, \quad \mathcal{A} = A \otimes I_M \tag{9}$$

where the entry $a_{sk} = 0$ if there is no edge connecting agents k and s. The matrix A is assumed to be symmetric and doubly stochastic (different from $\bar{A}$). We further assume the matrix to be primitive, i.e., there exists an integer $j > 0$ such that all entries of A^j are positive. Under these conditions it holds that $(I_{MK} - \mathcal{A})w = 0$ if and only if $w_k = w_s$ for all k, s (see [8], [10]).

C. Specific Instances

We start by rewriting recursion (8) in an equivalent manner by eliminating the dual variable y_i. From (8a) and (8b) it holds that

$$\begin{aligned} z_i - z_{i-1} = {} & (I - \mathcal{C})(w_{i-1} - w_{i-2}) - \mathcal{B}(y_{i-1} - y_{i-2}) \\ & - \mu\left(\nabla\mathcal{J}(w_{i-1}) - \nabla\mathcal{J}(w_{i-2})\right). \end{aligned} \tag{10}$$

From (8b) we know $\mathcal{B}(y_{i-1} - y_{i-2}) = \mathcal{B}^2 z_{i-1}$. Using this and rearranging the previous equation we get

$$\begin{aligned} z_i = {} & (I - \mathcal{B}^2)z_{i-1} + (I - \mathcal{C})(w_{i-1} - w_{i-2}) \\ & - \mu\left(\nabla\mathcal{J}(w_{i-1}) - \nabla\mathcal{J}(w_{i-2})\right). \end{aligned} \tag{11}$$

TABLE I
LISTING OF SOME STATE-OF-THE-ART FIRST-ORDER ALGORITHMS THAT CAN BE RECOVERED BY SPECIFIC CHOICES OF $\bar{\mathcal{A}}$, $\mathcal{B}$, AND $\mathcal{C}$ IN (8)

ATC algorithms	$\bar{\mathcal{A}}$	$\mathcal{B}^2$	$\mathcal{C}$
Aug-DGM [1], [2]	$\mathcal{A}^2$	$(I-\mathcal{A})^2$	0
ATC tracking [3], [4]	$\mathcal{A}$	$(I-\mathcal{A})^2$	$I-\mathcal{A}$
Exact diffusion [10]	$0.5(I+\mathcal{A})$	$0.5(I-\mathcal{A})$	0
NIDS [11]	$I-c(I-\mathcal{A})$	$c(I-\mathcal{A})$	0
NON-ATC algorithms	$\bar{\mathcal{A}}$	$\mathcal{B}^2$	$\mathcal{C}$
DIGing [6], [7]	I	$(I-\mathcal{A})^2$	$I-\mathcal{A}^2$
EXTRA [8]	I	$0.5(I-\mathcal{A})$	$0.5(I-\mathcal{A})$
DLM [9]	I	$c\mu\mathcal{L}$	$c\mu\mathcal{L}$

Matrix $\mathcal{A}$ is a typical symmetric and doubly stochastic network combination matrix introduced in (9). Matrix $\mathcal{L}$ is chosen such that the kth block of $\mathcal{L}w_i$ is equal to $\sum_{s\in\mathcal{N}_k} w_{k,i}-w_{s,i}$ and $c>0$ is a step-size parameter.

Using the above recursion, we will now choose specific matrices $\{\bar{\mathcal{A}},\mathcal{B},\mathcal{C}\}$ and show that we can recover many state-of-the-art algorithms (see Table I)

1) Exact Diffusion [10]: If we choose $\bar{\mathcal{A}}=0.5(I+\mathcal{A})$, $\mathcal{B}^2=0.5(I-\mathcal{A})$, and $\mathcal{C}=0$ in (11), multiply by $\bar{\mathcal{A}}$, and use $w_i=\bar{\mathcal{A}}z_i$ from (8c), we get

$$w_i=\bar{\mathcal{A}}(2w_{i-1}-w_{i-2}-\mu(\nabla\mathcal{J}(w_{i-1})-\nabla\mathcal{J}(w_{i-2}))). \tag{12}$$

The above recursion is the exact diffusion recursion first proposed in [10]. We also note that if we choose $\mathcal{C}=0$, $\mathcal{B}^2=c(I-\mathcal{A})$ ($c\in\mathbb{R}$), and $\bar{\mathcal{A}}=I-\mathcal{B}^2$ then we recover the smooth case of the NIDS algorithm from [11]. As highlighted in [11], NIDS is identical to exact diffusion for the smooth case when $c=0.5$.

2) Aug-DGM [1]: Substituting $\mathcal{B}=I-\mathcal{A}$ and $\mathcal{C}=0$ into (11), multiplying by $\bar{\mathcal{A}}=\mathcal{A}^2$, and using $w_i=\mathcal{A}^2z_i$ (8c), we get the recursion

$$w_i=\mathcal{A}(2w_{i-1}-\mathcal{A}w_{i-2}-\mu\mathcal{A}\left(\nabla\mathcal{J}(w_{i-1})-\nabla\mathcal{J}(w_{i-2})\right)). \tag{13}$$

The above recursion is equivalent to the Aug-DGM [1] (also known as ATC-DIGing [2]) algorithm

$$w_i=\mathcal{A}(w_{i-1}-\mu x_{i-1}) \tag{14a}$$

$$x_i=\mathcal{A}\left(x_{i-1}+\nabla\mathcal{J}(w_i)-\nabla\mathcal{J}(w_{i-1})\right). \tag{14b}$$

By eliminating the gradient tracking variable x_i, we can rewrite the previous recursion as (13) (see Appendix B).

3) ATC Tracking Method [3], [4]: Substituting $\mathcal{B}=I-\mathcal{A}$ and $\mathcal{C}=I-\mathcal{A}$ into (11), multiplying by $\bar{\mathcal{A}}=\mathcal{A}$, and using $w_i=\mathcal{A}z_i$ from (8c), we get the recursion

$$w_i=\mathcal{A}(2w_{i-1}-\mathcal{A}w_{i-2}-\mu(\nabla\mathcal{J}(w_{i-1})-\nabla\mathcal{J}(w_{i-2}))). \tag{15}$$

The above recursion is equivalent to the following variant of the ATC tracking method [3], [4]:

$$w_i=\mathcal{A}(w_{i-1}-\mu x_{i-1}) \tag{16a}$$

$$x_i=\mathcal{A}x_{i-1}+\nabla\mathcal{J}(w_i)-\nabla\mathcal{J}(w_{i-1}). \tag{16b}$$

By eliminating the gradient tracking variable x_i, we can show that the previous recursion is exactly (15) (see Appendix B).

4) NON-ATC Algorithms: We note that DIGing [6], [7], EXTRA [8], and the decentralized linearized alternating direction method of multipliers (DLM) [9] can also be represented by (8) with $\bar{\mathcal{A}}=I$ and proper choices of $\mathcal{B}^2$ and $\mathcal{C}$ (see Table I). Since $\bar{\mathcal{A}}=I$, these algorithms are not of the ATC form. Due to space constraints, we only focus on the ATC form in this article and refer interested readers to the extended technical report [29] for the non-ATC form.

Remark 1 (Communication Cost): Note that exact diffusion (12) requires one round of communication (or combination) per iteration. This means that each agent sends an M vector to its neighbor per iteration. On the other hand, the gradient tracking method (16) requires two rounds of combination/communication per iteration for the vectors $w_{i-1}-\mu x_{i-1}$ and x_{i-1}, which means each agent sends a $2M$ vector to its neighbor. Similarly, the Aug-DGM (ATC-DIGing) method (14) also requires two rounds of combination per iteration for the vectors $w_{i-1}-\mu x_{i-1}$ and $x_{i-1}+\nabla\mathcal{J}(w_i)-\nabla\mathcal{J}(w_{i-1})$; moreover, it requires communicating these two variables sequentially (at different communication steps). □

III. PROXIMAL UNIFIED DECENTRALIZED ALGORITHM (PUDA)

In this section, we extend UDA (8) to handle the nondifferentiable component $R(w)$ to get a proximal unified decentralized algorithm (PUDA). Let us introduce the network quantity $\mathcal{R}(w)\triangleq\frac{1}{K}\sum_{k=1}^K R(w_k)$. With this definition, we propose the following recursion: Let $y_{-1}=0$ and w_{-1} take any arbitrary value. Repeat for $i=0,1,\ldots$

$$\begin{cases} z_i=(I-\mathcal{C})w_{i-1}-\mu\nabla\mathcal{J}(w_{i-1})-\mathcal{B}y_{i-1} & \text{(17a)}\\ y_i=y_{i-1}+\mathcal{B}z_i & \text{(17b)}\\ w_i=\mathbf{prox}_{\mu\mathcal{R}}\left(\bar{\mathcal{A}}z_i\right). & \text{(17c)}\end{cases}$$

We refer the reader to Appendix A for specific instances of PUDA (17) and how to implement them in a decentralized manner. In the following, we will show that w_i in the above recursion converges to $\mathbb{1}_K\otimes w^\star$ where $w^\star$ is the desired solution of (1). We first prove the existence and optimality of the fixed points of recursion (17).

Lemma 1 (Optimality Point): *Under Assumption* 1 *and condition (5), a fixed point* $(w^\star,y^\star,z^\star)$ *exists for recursions* (17a)–(17c), *i.e., it holds that*

$$\begin{cases} z^\star=\ w^\star-\mu\nabla\mathcal{J}(w^\star)-\mathcal{B}y^\star & \text{(18a)}\\ 0=\ \mathcal{B}z^\star & \text{(18b)}\\ w^\star=\ \mathbf{prox}_{\mu\mathcal{R}}(\bar{\mathcal{A}}z^\star). & \text{(18c)}\end{cases}$$

Moreover, $w^\star$ *and* $z^\star$ *are unique with* $w^\star=\mathbb{1}_K\otimes w^\star$, *where* $w^\star$ *is the solution of problem* (1).

Proof: The proof adjusts the arguments from [20] to the general framework (17), if desired, see the extended technical report [29] for the argument. □

IV. LINEAR CONVERGENCE

Note that there exists a particular fixed point $(w^\star,y_b^\star,z^\star)$, where $y_b^\star$ is a unique vector that belongs to the range space of $\mathcal{B}$—see [20, Remark 2]. In the following we will show that the iterates (w_i,y_i,z_i) converge linearly to this particular fixed point $(w^\star,y_b^\star,z^\star)$. To this end, we introduce the error quantities

$$\widetilde{w}_i\triangleq w_i-w^\star,\quad \widetilde{y}_i\triangleq y_i-y_b^\star,\quad \widetilde{z}_i\triangleq z_i-z^\star. \tag{19}$$

Note that from condition (5) we have $\mathcal{C}w^\star=0$. Therefore, from (17a)–(17c) and (18a)–(18c) we can reach the following error recursions:

$$\widetilde{z}_i=(I-\mathcal{C})\widetilde{w}_{i-1}-\mu\left(\nabla\mathcal{J}(w_{i-1})-\nabla\mathcal{J}(w^\star)\right)-\mathcal{B}\widetilde{y}_{i-1} \tag{20a}$$

$$\widetilde{y}_i=\widetilde{y}_{i-1}+\mathcal{B}\widetilde{z}_i \tag{20b}$$

$$\widetilde{w}_i=\mathbf{prox}_{\mu\mathcal{R}}\left(\bar{\mathcal{A}}z_i\right)-\mathbf{prox}_{\mu\mathcal{R}}(\bar{\mathcal{A}}z^\star). \tag{20c}$$

For our convergence result, we need the following technical conditions.

Assumption 2 (Consensus Matrices) *It is assumed that both condition (5) and the following condition hold:*

$$\bar{\mathcal{A}}^2 \leq I - \mathcal{B}^2 \text{ and } 0 \leq \mathcal{C} < 2I. \tag{21}$$

□

Remark 2 (Convergence Conditions): Note that the above conditions are satisfied for exact diffusion [30] and NIDS [11]. For the ATC tracking methods (14) and (16), the conditions translate to the requirement that the eigenvalues of A are between [0,1], rather than the typical $(-1, 1]$. Although this condition is not necessary, it can be easily satisfied by redefining $A \leftarrow 0.5(I + A)$. We also impose it to unify the analysis of these methods through a short proof. Note that most works that analyze decentralized methods under more relaxed conditions on the network topology impose restrictive step-size conditions that depend on the network and on the order of $O(\nu^{\theta_1}/\delta^{\theta_2})$, where $0 < \theta_1 \leq 1$ and $\theta_2 > 1$ (see [2], [6], [12], [15]). On the other hand, we require step sizes of order $O(1/\delta)$. Moreover, we will show that any algorithm that fits into our setup with $\mathcal{C} = 0$ can use a step size as large as the centralized proximal gradient descent (see discussion after Theorem 1). □

Remark 3 (Non-ATC Conditions): Assumption 2 is used to analyze the ATC structure, where $\bar{A}$ is a primitive combination matrix. The non-ATC structure with $\bar{A} = I$ can also be analyzed under slightly different conditions. See extended technical report [29] for the non-ATC analysis. □

Note that $\mathcal{B}^2$ and $\mathcal{C}$ are symmetric; thus, their singular values are equal to their eigenvalues. Moreover, since the square of a symmetric matrix is positive semidefinite, Assumption 2 implies $0 < \underline{\sigma}(\mathcal{B}^2) \leq 1$ and $\sigma_{\max}(\mathcal{C}) < 2$.

Theorem 1 (Linear Convergence): *Under Assumptions* 1–2, *if* $y_0 = 0$ *and the step size satisfies* $\mu < \frac{2-\sigma_{\max}(\mathcal{C})}{\delta}$, *it holds that* $\|\widetilde{w}_i\|^2 + \|\widetilde{y}_i\|^2 \leq \gamma(\|\widetilde{w}_{i-1}\|^2 + \|\widetilde{y}_{i-1}\|^2)$, *where*

$$\gamma = \max\left\{1 - \mu\nu(2 - \sigma_{\max}(\mathcal{C}) - \mu\delta), 1 - \underline{\sigma}(\mathcal{B}^2)\right\} < 1. \tag{22}$$

Proof: Squaring both sides of (20a) and (20b) we get

$$\begin{aligned}\|\widetilde{z}_i\|^2 = {} & \|(I-\mathcal{C})\widetilde{w}_{i-1} - \mu\left(\nabla\mathcal{J}(w_{i-1}) - \nabla\mathcal{J}(w^\star)\right)\|^2 + \|\mathcal{B}\widetilde{y}_{i-1}\|^2 \\ & - 2\widetilde{y}_{i-1}^{\mathsf{T}}\mathcal{B}((I-\mathcal{C})\widetilde{w}_{i-1} - \mu(\nabla\mathcal{J}(w_{i-1}) - \nabla\mathcal{J}(w^\star)))\end{aligned} \tag{23}$$

and

$$\begin{aligned}\|\widetilde{y}_i\|^2 = {} & \|\widetilde{y}_{i-1}\|^2 + \|\mathcal{B}\widetilde{z}_i\|^2 + 2\widetilde{y}_{i-1}^{\mathsf{T}}\mathcal{B}\widetilde{z}_i \\ \overset{(20a)}{=} {} & \|\widetilde{y}_{i-1}\|^2 + \|\widetilde{z}_i\|_{\mathcal{B}^2}^2 - 2\|\mathcal{B}\widetilde{y}_{i-1}\|^2 \\ & + 2\widetilde{y}_{i-1}^{\mathsf{T}}\mathcal{B}((I-\mathcal{C})\widetilde{w}_{i-1} - \mu\left(\nabla\mathcal{J}(w_{i-1}) - \nabla\mathcal{J}(w^\star)\right)).\end{aligned} \tag{24}$$

Adding (24) to (23) and rearranging, we get

$$\begin{aligned}& \|\widetilde{z}_i\|_{\mathcal{Q}}^2 + \|\widetilde{y}_i\|^2 \\ & = \|(I-\mathcal{C})\widetilde{w}_{i-1} - \mu\left(\nabla\mathcal{J}(w_{i-1}) - \nabla\mathcal{J}(w^\star)\right)\|^2 \\ & \quad + \|\widetilde{y}_{i-1}\|^2 - \|\mathcal{B}\widetilde{y}_{i-1}\|^2 \\ & \leq \left\|\widetilde{w}_{i-1} - \mu\left(\nabla\mathcal{J}(w_{i-1}) - \nabla\mathcal{J}(w^\star) + \frac{1}{\mu}\mathcal{C}\widetilde{w}_{i-1}\right)\right\|^2 \\ & \quad + \left(1 - \underline{\sigma}(\mathcal{B}^2)\right)\|\widetilde{y}_{i-1}\|^2\end{aligned} \tag{25}$$

where $\mathcal{Q} = I - \mathcal{B}^2$ is positive semidefinite from (21). The last inequality holds since for $y_0 = 0$ and $y_i = y_{i-1} + \mathcal{B}z_i$, we know $y_i \in \text{range}(\mathcal{B})$ for any i. Thus, both y_i and $y_b^\star$ lie in the range space of $\mathcal{B}$, and it holds that $\|\mathcal{B}\widetilde{y}_{i-1}\|^2 \geq \underline{\sigma}(\mathcal{B}^2)\|\widetilde{y}_{i-1}\|^2$. Also, since $\mathcal{J}(w) + \frac{1}{2\mu}\|w\|_{\mathcal{C}}^2$ is $\delta_\mu = \delta + \frac{1}{\mu}\sigma_{\max}(\mathcal{C})$ smooth, it holds that [31, Th. 2.1.5]

$$\begin{aligned}& \|\nabla\mathcal{J}(w_{i-1}) - \nabla\mathcal{J}(w^\star) + \frac{1}{\mu}\mathcal{C}\widetilde{w}_{i-1}\|^2 \\ & \leq \delta_\mu\widetilde{w}_{i-1}^{\mathsf{T}}\left(\nabla\mathcal{J}(w_{i-1}) - \nabla\mathcal{J}(w^\star) + \frac{1}{\mu}\mathcal{C}\widetilde{w}_{i-1}\right).\end{aligned} \tag{26}$$

Using this bound, it can be easily verified that

$$\begin{aligned}& \|\widetilde{w}_{i-1} - \mu\left(\nabla\mathcal{J}(w_{i-1}) - \nabla\mathcal{J}(w^\star) + \frac{1}{\mu}\mathcal{C}\widetilde{w}_{i-1}\right)\|^2 \leq \|\widetilde{w}_{i-1}\|^2 \\ & \quad - \mu(2-\mu\delta_\mu)\widetilde{w}_{i-1}^{\mathsf{T}}\left(\nabla\mathcal{J}(w_{i-1}) - \nabla\mathcal{J}(w^\star) + \frac{1}{\mu}\mathcal{C}\widetilde{w}_{i-1}\right) \\ & \leq \left(1 - \mu\nu(2-\mu\delta_\mu)\right)\|\widetilde{w}_{i-1}\|^2\end{aligned} \tag{27}$$

where in the last step we used the fact that $2 - \mu\delta_\mu > 0$, which follows from the condition $\mu < (2 - \sigma_{\max}(\mathcal{C}))/\delta$, and the fact that $\mathcal{J}(w) + \frac{1}{2\mu}\|w\|_{\mathcal{C}}^2$ is ν-strongly convex. Thus, we can substitute the previous inequality in (25) and get

$$\begin{aligned}\|\widetilde{z}_i\|_{\mathcal{Q}}^2 + \|\widetilde{y}_i\|^2 \leq {} & \left(1 - \mu\nu(2 - \sigma_{\max}(\mathcal{C}) - \mu\delta)\right)\|\widetilde{w}_{i-1}\|^2 \\ & + \left(1 - \underline{\sigma}(\mathcal{B}^2)\right)\|\widetilde{y}_{i-1}\|^2.\end{aligned} \tag{28}$$

From (20c) and the nonexpansive property of the proximal operator, we have

$$\begin{aligned}\|\widetilde{w}_i\|^2 &= \|\mathbf{prox}_{\mu\mathcal{R}}\left(\bar{\mathcal{A}}z_i\right) - \mathbf{prox}_{\mu\mathcal{R}}(\bar{\mathcal{A}}z^\star)\|^2 \\ &\leq \|\bar{\mathcal{A}}\widetilde{z}_i\|^2 \leq \|\widetilde{z}_i\|_{\mathcal{Q}}^2.\end{aligned} \tag{29}$$

where the last step holds because of condition (21), so that $\|\bar{\mathcal{A}}\widetilde{z}_i\|^2 = \|\widetilde{z}_i\|_{\bar{\mathcal{A}}^2}^2 \leq \|\widetilde{z}_i\|_{\mathcal{Q}}^2$. Substituting (29) into (28) we reach our result. □

An interesting choice of $\bar{\mathcal{A}}$, $\mathcal{B}$, and $\mathcal{C}$ is the class with $\mathcal{C} = 0$. For $\mathcal{C} = 0$, which is the case for exact diffusion (12) and Aug-DGM (ATC-DIGing) (14), the step-size bound in Theorem 1 becomes $\mu < \frac{2}{\delta}$, which is independent of the network and as large as the centralized proximal gradient descent. Moreover, for $\mathcal{C} = 0$ the convergence rate becomes $\gamma = \max\{1 - \mu\nu(2 - \mu\delta), 1 - \underline{\sigma}(\mathcal{B}^2)\} < 1$, which separates the network effect from the cost function. If we further choose $\bar{\mathcal{A}} = \mathcal{A}^j$ and $\mathcal{B}^2 = I - \mathcal{A}^j$ for integer $j \geq 1$, then we have $1 - \underline{\sigma}(\mathcal{B}^2) = \lambda_2(\mathcal{A}^j) \to 0$ as $j \to \infty$, where $\lambda_2(\mathcal{A}^j)$ is the second largest eigenvalue of $\mathcal{A}^j$. Thus, the convergence rate $\gamma = 1 - \mu\nu(2 - \mu\delta)$ can match the rate of centralized algorithms for large enough j. A similar conclusion appears for NIDS [11] but for the smooth case, which is subsumed in our framework.

Remark 4 (Network Effect): The convergence rate depends on the network graph through the terms $\underline{\sigma}(\mathcal{B}^2)$ and $\sigma_{\max}(\mathcal{C})$. Given a certain graph, it will depend on the number of agents K *indirectly* as we now explain. If we choose $\mathcal{B}^2 = I - \mathcal{A}$ and $\mathcal{C} = 0$, where $\mathcal{A}$ is constructed as in Section II-B and satisfy Assumption 2. Then, we have that $1 - \underline{\sigma}(\mathcal{B}^2) = \lambda_2(\mathcal{A})$, where $\lambda_2(\mathcal{A})$ denotes the second largest eigenvalue of $\mathcal{A}$. For a cyclic network it holds that $\lambda_2(\mathcal{A}) = 1 - \mathcal{O}(1/K^2)$. For a grid network we have $\lambda_2(\mathcal{A}) = 1 - \mathcal{O}(1/K)$. For a fully connected network, we can choose $\mathcal{A} = \frac{1}{K}\mathbb{1}\mathbb{1}^{\mathsf{T}}$, so that $\lambda_2(\mathcal{A}) = 0$. In this case, we can also choose $\bar{\mathcal{A}} = \frac{1}{K}\mathbb{1}\mathbb{1}^{\mathsf{T}}$ and the primal updates in (17) becomes, so that each agent updates its vector via a proximal gradient descent update on the objective function given in problem (1). □

V. Simulations on Real Data

In this section we test the performance of three different instances of the proposed method (17) along with some state-of-the-art algorithms.

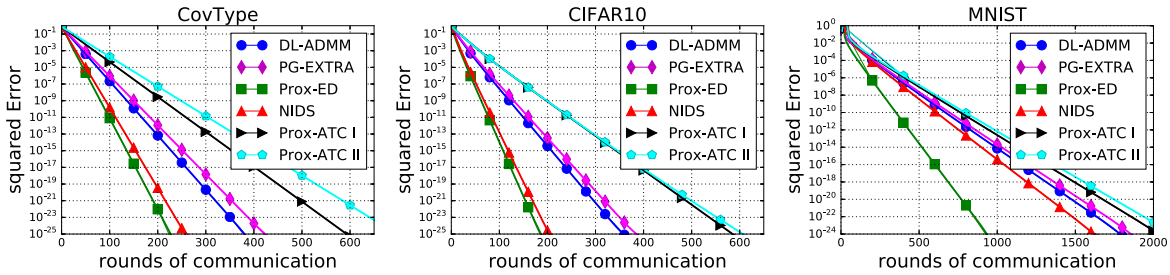

Fig. 1. Simulation results. The y-axis indicates the relative squared error $\sum_{k=1}^{K}\|w_{k,i}-w^\star\|^2/\|w^\star\|^2$. Prox-ED refers to (17) with $\bar{\mathcal{A}}=0.5(I+\mathcal{A})$, $\mathcal{B}^2=0.5(I-\mathcal{A})$, and $\mathcal{C}=0$. Prox-ATC I refers to (17) with $\bar{\mathcal{A}}=\mathcal{A}^2$, $\mathcal{B}=I-\mathcal{A}$, and $\mathcal{C}=0$. Prox-ATC II refers to (17) with $\bar{\mathcal{A}}=\mathcal{A}$, $\mathcal{B}=I-\mathcal{A}$, and $\mathcal{C}=I-\mathcal{A}$. DL-ADMM [22], PG-EXTRA [21], NIDS [11].

We consider problem (1) with

$$J_k(w)=\frac{1}{L}\sum_{\ell=1}^{L}\ln(1+\exp(-y_{k,\ell}x_{k,\ell}^{\mathsf{T}}w))+\frac{\lambda}{2}\|w\|^2$$

and $R_k(w)=\rho\|w\|_1$, where $\{x_{k,\ell},y_{k,\ell}\}_{\ell=1}^{L}$ are local data kept by agent k and L is the size of the local dataset. We consider three real datasets: Covtype.binary, MNIST, and CIFAR10. The last two datasets have been transformed into binary classification problems by considering data with two labels, digits two and four ("2" and "4") classes for MNIST, and cat and dog classes for CIFAR-10. In Covtype.binary we use 50 000 samples as training data and each data has dimension 54. In MNIST we use 10 000 samples as training data and each data has dimension 784. In CIFAR-10 we use 10 000 training data and each data has dimension 3072. All features have been preprocessed and normalized to the unit vector with sklearn's normalizer.[3] For the network, we generated a randomly connected network with $K=20$ agents (see [29, Fig. 1]). The associated combination matrix A is generated according to the Metropolis rule [19]. For all simulations, we assign data evenly to each agent. We set $\lambda=10^{-4}$ and $\rho=2\times10^{-3}$ for Covtype, $\lambda=10^{-2}$ and $\rho=5\times10^{-4}$ for CIFAR-10, and $\lambda=10^{-4}$ and $\rho=2\times10^{-3}$ for MNIST. The simulation results are shown in Fig. 1. The decentralized implementations of Prox-ED, Prox-ATC I, and Prox-ATC II are given in Appendix A. For each algorithm, we tune the step sizes manually to achieve the best possible convergence rate. We notice that the performance of each algorithm differs in each dataset and Prox-ED performs the best in our simulation setup. The x-axis in these plots is in terms of rounds of communication per iteration. Note that Prox-ATC I and Prox-ATC II require two rounds of communication per iteration compared to only one round for all other algorithms, see Remark 1.

VI. Separate Nonsmooth Terms: Sublinear Rate

In this section, we will show that if each agent owns a different local nonsmooth term, then *exact* global linear convergence cannot be attained in the worst case (for all problem instances) although it can still be possible for some special cases. Consider the more general problem with agent specific regularizers

$$\min_{w\in\mathbb{R}^M}\ \frac{1}{K}\sum_{k=1}^{K}J_k(w)+R_k(w) \tag{30}$$

where $J_k(w)$ is a strongly convex smooth function and $R_k(w)$ is nonsmooth convex with closed form proximal mappings (each $J_k(w)$ and $R_k(w)$ are further assumed to be closed and proper functions).

[3]https://scikit-learn.org

Although many algorithms (centralized and decentralized) exist that solve (30), none have been shown to achieve linear convergence in the presence of general nonsmooth proximal terms $R_k(w)$. In the following, by tailoring the results from [25], we show that this is not possible when having access to the proximal mapping of each individual nonsmooth term $R_k(w)$ separately.

A. Sublinear Lower Bound

Let $\mathcal{H}$ be a deterministic algorithm that queries

$$\{J_k(\cdot),R_k(\cdot),\nabla J_k(\cdot),\mathbf{prox}_{\mu_{i,k}R_k}(\cdot)\,|\,\mu_{i,k}>0\}$$

for all $k=1,\dots,K$ once for each iteration $i=0,1,\dots$. To clarify, the scalar parameter $\mu_{i,k}>0$ can differ for $i=0,1,\dots$ and $k=1,\dots,K$ or they can be constants (e.g., $\mu_{i,k}=\mu>0$). Note that $\mathcal{H}$ has the option to combine the queried values in any possible combination (e.g., it can only use certain information from certain communications). Thus, $\mathcal{H}$ includes decentralized algorithms in which communication is restricted to edges on a graph.

Consider the specific instance of (30)

$$\min_{w\in\mathbb{R}^M}\ F_\nu(w)=\frac{\nu}{2}\|w\|^2+\frac{1}{K}\sum_{k=1}^{K}R_k(w) \tag{31}$$

where $\nu>0$ and $J_k(w)=\frac{\nu}{2K}\|w\|^2$. Assume $R_k(w)<\infty$ if and only if $\|w\|\le B$ and $|R_k(w_1)-R_k(w_2)|\le G\|w_1-w_2\|$ for all w_1,w_2 (where B and G are some positive constants), such that $\|w_1\|\le B$ and $\|w_2\|\le B$. To prove that linear convergence is not possible, we will reduce our setup to $\min_{w\in\mathbb{R}^M}F_0(w)$, which has a known lower bound [25]. Let $\mathcal{H}_o$ be a deterministic algorithm that queries

$$\{R_k(\cdot),\mathbf{prox}_{\mu_{i,k}R_k(\cdot)}(\cdot)\,|\,\mu_{i,k}>0,\ k=1,\dots,K\}$$

once for each iteration $i=0,1,\dots$ and communicates through a fully connected network. The following result is a special case of the more general result [25, Th. 1].

Theorem 2: *Let $0<B$, $0<G$, $2\le K$, and $0<\varepsilon<GB/12$. For a large enough problem dimension $M=\mathcal{O}(KGB/\varepsilon)$, the algorithm $\mathcal{H}_o$ (in the worst case) requires $\mathcal{O}(GB/\varepsilon)$ or more iterations to find a $\hat{w}$, such that $F_0(\hat{w})-\inf_w F_0(w)<\varepsilon$.*

We argue that algorithm $\mathcal{H}$ cannot be too efficient at solving $\min_w F_\nu(w)$ with $\nu>0$ as otherwise it can be used to efficiently solve $\min_w F_0(w)$ and contradict Theorem 2.

Theorem 3: *Let $0<\nu$, $0<B$, $0<G$, $2\le K$, and $0<\varepsilon<G^2/(288\nu)$. For a large enough problem dimension $M=\mathcal{O}(KG/\sqrt{\nu\varepsilon})$, the algorithm $\mathcal{H}$ (in the worst case) requires $\mathcal{O}(G/\sqrt{\nu\varepsilon})$ or more iterations to find a $\hat{w}$, such that $F_\nu(\hat{w})-\inf_w F_\nu(w)<\varepsilon$.*

Proof: This argument modifies the proof of [25, Th. 2], which makes a similar but slightly different claim. Let $\nu = \varepsilon/B^2$ and $w_\nu^\star$ denotes the minimizer of F_ν. Assume for contradiction that $\mathcal{H}$ can find a $\hat{w}$, such that

$$F_\nu(\hat{w}) - F_\nu(w_\nu^\star) < \frac{\varepsilon}{2} \tag{32}$$

in $o(G/\sqrt{\nu\varepsilon})$ iterations. Note that for all w, such that $\|w\| \leq B$, it holds from (31)

$$F_\nu(w) \leq F_0(w) + \frac{\nu B^2}{2} = F_0(w) + \frac{\varepsilon}{2}. \tag{33}$$

Putting these together, we get

$$\begin{aligned} F_0(\hat{w}) - F_0(w_0^\star) - \frac{\varepsilon}{2} &\overset{(33)}{\leq} F_0(\hat{w}) - F_\nu(w_0^\star) \\ &\overset{(a)}{\leq} F_\nu(\hat{w}) - F_\nu(w_\nu^\star) < \frac{\varepsilon}{2} \end{aligned} \tag{34}$$

where in step (a) we used $F_0(w) \leq F_\nu(w)$ and $F_\nu(w_\nu^\star) \leq F_\nu(w_0^\star)$. We conclude that $F_0(\hat{w}) - F_0(w_0^\star) < \varepsilon$. Since $\nabla J_k(\cdot) = \frac{\nu}{K} I$ is just a scaled identity, querying $\nabla J_k(\cdot)$ does not provide a new direction that $\mathcal{H}_o$ could otherwise not use. Thus, algorithm $\mathcal{H}$ applied to minimizing F_ν is an instance of algorithm $\mathcal{H}_o$. This means that we have an algorithm for minimizing F_0 in $o(G/\sqrt{\nu\varepsilon}) = o(GB/\varepsilon)$ iterations, which contradicts Theorem 2. Note that $0 < \varepsilon < GB/12$ from Theorem 2 and by using $\nu = \varepsilon/B^2$ we require $0 < \varepsilon < G^2/(288\nu)$ [an extra factor of 2 appears because of (32)]. □

Remark 5 (Dimension Independent Bound): The lower bound of Theorem 3 is *dimension independent* in the same way other Nesterov-type lower bounds are [25] and [31]. The result implies that it is not possible to establish linear convergence of $\mathcal{H}$ with a rate depending on ν and G, but not on the problem dimension M. That said, a dimension dependent linear convergence may be established. For example, *eventual linear convergence*[4] has been established in [32] when the functions $\{J_k(\cdot), R_k(\cdot)\}$ are piecewise linear quadratic. This result does not contradict our result as the linear rate and the number of iterations needed to observe the linear rate are *dependent* on the problem dimension. Our linear convergence result of Theorem 1 is dimension independent as it holds for any dimension M. □

B. Numerical Counterexample

In this section, we numerically show that linear convergence to the exact solution $w^\star$ is not possible in general. We consider an instance of (30) with $K = 2$, M is a very large even number, and quadratic smooth terms $J_k(w) = \eta/2\|w\|^2$ for some $\eta > 0$. We let the nonsmooth terms be

$$\begin{aligned} R_1(w) &= |\sqrt{2}w(1) - 1| + |w(2) - w(3)| + |w(4) - w(5)| \\ &\quad + \cdots + |w(M-2) - w(M-1)| \end{aligned} \tag{35a}$$

$$\begin{aligned} R_2(w) &= |w(1) - w(2)| + |w(3) - w(4)| \\ &\quad + \cdots + |w(M-1) - w(M)|. \end{aligned} \tag{35b}$$

Both $\mathbf{prox}_{R_1}$ and $\mathbf{prox}_{R_2}$ have closed forms (see [29]). The above function structure is related to the one in [26], which was used to derive lower bounds for a different class of algorithms as explained in the introduction.

[4]A sequence $\{x_i\}_{i=0}^\infty$ has eventual linear convergence to $x^\star$ if there exists a sufficiently large i_o, such that $\|x_i - x^\star\| \leq \gamma^i C$ for some $C > 0$ and all $i \geq i_o$.

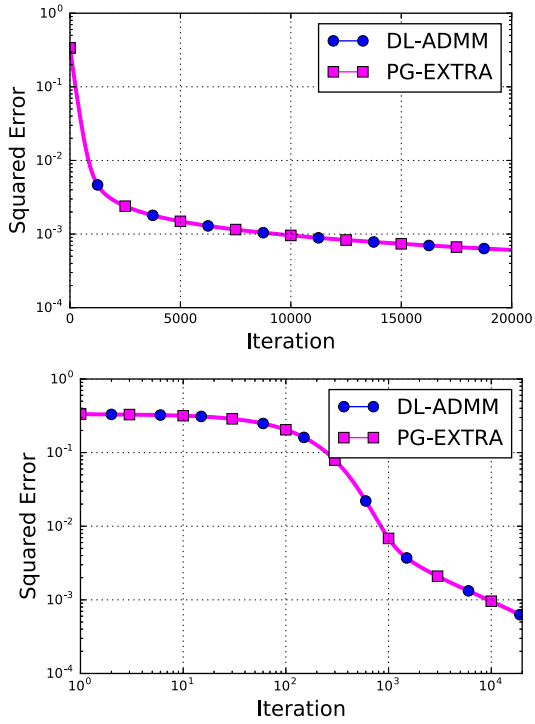

Fig. 2. Numerical counterexample simulations. Both y-axis and x-axis are in logarithmic scales in the bottom figure. PG-EXTRA [21] and DL-ADMM [22], [23] converge sublinearly to the solution of the proposed numerical counterexample.

In the numerical experiment, we test the performance of two well-known decentralized proximal methods, PG-EXTRA [21] and DL-ADMM [22], [23]. Note that the structure of updates (17) are designed to handle a common nonsmooth term case only, which is why we do not test it in this numerical counterexample. We set $M = 2000$ and $\eta = 1$. The step sizes for both PG-EXTRA and DL-ADMM are set to 0.005. The combination matrix is set as $A = \frac{1}{2}\mathbb{1}_2\mathbb{1}_2^\mathsf{T}$. The numerical results in the top plot of Fig. 2 shows that both PG-EXTRA and DL-ADMM converge sublinearly to the solution. In particular, we see that the error curves after around 10^3 iterations have sublinear convergence. The bottom plot in Fig. 2 shows the squared error where both x-axis and y-axis are in logarithmic scales. In this scale, a straight line indicates a sublinear rate, which is clearly visible after around 10^3 iterations. No global linear convergence is observed in the simulation for sufficiently large dimension M and algorithms independent of M, which is consistent with our discussion in Remark 5.

VII. Conclusion

In this article, we proposed a proximal primal-dual algorithmic framework, which subsumes many existing algorithms in the smooth case, and established its linear convergence under strongly convex objectives. Our analysis provides wider step-size conditions than many existing works, which provides insightful indications on the performance of each algorithm. That said, these step-size bound comes at the expense of stronger assumption on the combination matrices, see Remark 2. It is therefore of interest to study the interrelation between the step sizes and combination matrices for linear convergence. Regarding the discussion below Theorem 1, a useful future direction is to study how to optimally choose $\bar{\mathcal{A}}$, $\mathcal{B}$, and $\mathcal{C}$ as a function of $\mathcal{A}$ to get the best possible convergence rate while balancing the communication cost per iteration.

APPENDIX A
IMPLEMENTATION OF (17)

A. Prox-ED: $\bar{\mathcal{A}} = 0.5(I + \mathcal{A})$, $\mathcal{B}^2 = 0.5(I - \mathcal{A})$, and $\mathcal{C} = 0$

Recursion (17) with $\bar{\mathcal{A}} = 0.5(I + \mathcal{A}), \mathcal{B}^2 = 0.5(I - \mathcal{A})$, and $\mathcal{C} = 0$ is equivalent to the proximal exact diffusion (Prox-ED) recursion listed in (40a)–(40d). To see this, note that for $i = 0$, it is straight forward to check that each block in (17c) is the same as $w_{k,0}$ in (40). Now we will show the equivalence for $i \geq 1$. From (17a), we know that

$$\begin{aligned} &z_i - z_{i-1} \\ &= w_{i-1} - w_{i-2} - \mu\left(\nabla\mathcal{J}(w_{i-1}) - \nabla\mathcal{J}(w_{i-2})\right) - \mathcal{B}(y_{i-1} - y_{i-2}) \\ &= w_{i-1} - w_{i-2} - \mu\left(\nabla\mathcal{J}(w_{i-1}) - \nabla\mathcal{J}(w_{i-2})\right) - \mathcal{B}^2 z_{i-1} \end{aligned} \tag{36}$$

where we used (17b) in the last step. Rearranging and noting that $\mathcal{B}^2 = 0.5(I - \mathcal{A})$ we get

$$z_i = \bar{\mathcal{A}} z_{i-1} + w_{i-1} - w_{i-2} - \mu\left(\nabla\mathcal{J}(w_{i-1}) - \nabla\mathcal{J}(w_{i-2})\right). \tag{37}$$

By multiplying $\bar{\mathcal{A}}$ to both sides of the previous equation and introducing $x_i \triangleq \bar{\mathcal{A}} z_i$ we get

$$x_i = \bar{\mathcal{A}}(x_{i-1} + w_{i-1} - w_{i-2} - \mu\left(\nabla\mathcal{J}(w_{i-1}) - \nabla\mathcal{J}(w_{i-2})\right)).$$

Thus from (17c) we get

$$\begin{cases} x_i = \bar{\mathcal{A}}(x_{i-1} + w_{i-1} - w_{i-2} - \mu\left(\nabla\mathcal{J}(w_{i-1}) - \nabla\mathcal{J}(w_{i-2})\right)) \\ w_i = \mathbf{prox}_{\mu R}(x_i). \end{cases}$$

The above recursion is equivalent to (40a)–(40d). This can be easily seen by substituting (40a)–(40b) into (40c).

B. Prox-ATC I: $\bar{\mathcal{A}} = \mathcal{A}^2$, $\mathcal{B}^2 = (I - \mathcal{A})^2$, and $\mathcal{C} = 0$

For the choice $\bar{\mathcal{A}} = \mathcal{A}^2$, $\mathcal{B}^2 = (I - \mathcal{A})^2$, and $\mathcal{C} = 0$, we can represent (17) as listed in (41). This can be seen by following the same approach as the previous subsection. To see this, note that with $\mathcal{B}^2 = (I - \mathcal{A})^2$ we will get

$$\begin{aligned} z_i = (2\mathcal{A} - \mathcal{A}^2) z_{i-1} + w_{i-1} - w_{i-2} \\ - \mu\left(\nabla\mathcal{J}(w_{i-1}) - \nabla\mathcal{J}(w_{i-2})\right). \end{aligned} \tag{38}$$

By multiplying $\mathcal{A}^2$ to both sides of the previous equation and introducing $x_i \triangleq \mathcal{A}^2 z_i$ we get

$$\begin{aligned} x_i = \mathcal{A}((2I - \mathcal{A}) x_{i-1} + \mathcal{A} w_{i-1} - \mathcal{A} w_{i-2} \\ - \mu\mathcal{A}\left(\nabla\mathcal{J}(w_{i-1}) - \nabla\mathcal{J}(w_{i-2})\right)). \end{aligned} \tag{39}$$

Thus from (17c) we have $w_i = \mathbf{prox}_{\mu R}(x_i)$.

C. Prox-ATC II: $\bar{\mathcal{A}} = \mathcal{A}$, $\mathcal{B} = I - \mathcal{A}$, and $\mathcal{C} = I - \mathcal{A}$

For the choice $\bar{\mathcal{A}} = \mathcal{A}$, $\mathcal{B} = I - \mathcal{A}$, and $\mathcal{C} = I - \mathcal{A}$, we can represent (17) as listed in (42). This can be seen by following the same approach as the previous subsection, see [29] for details.

APPENDIX B
EQUIVALENT REPRESENTATION

A. Aug-DGM (ATC-DIGing)

Here we show that (14) is equivalent to (13). From (14a) we have

$$\begin{aligned} &w_i - \mathcal{A} w_{i-1} \\ &= \mathcal{A}(w_{i-1} - \mathcal{A} w_{i-2} - \mu\left(x_{i-1} - \mathcal{A} x_{i-2}\right)) \\ &\overset{(14b)}{=} \mathcal{A}(w_{i-1} - \mathcal{A} w_{i-2} - \mu\mathcal{A}\left(\nabla\mathcal{J}(w_{i-1}) - \nabla\mathcal{J}(w_{i-2})\right)). \end{aligned}$$

Algorithm 1: (Prox-ED).

Setting: Let $\bar{A} = [\bar{a}_{sk}] = (I_K + A)/2$. Initialize $x_{k,-1} = \psi_{k,-1}$ and $w_{k,-1}$ arbitrary. For every agent k, repeat for $i = 0, 1, 2, \ldots$

$$\psi_{k,i} = w_{k,i-1} - \mu\nabla J_k(w_{k,i-1}) \tag{40a}$$

$$z_{k,i} = x_{k,i-1} + \psi_{k,i} - \psi_{k,i-1} \tag{40b}$$

$$x_{k,i} = \sum_{s\in\mathcal{N}_k} \bar{a}_{sk} z_{s,i} \tag{40c}$$

$$w_{k,i} = \mathbf{prox}_{\mu R}(x_{k,i}) \tag{40d}$$

Algorithm 2: Prox-ATC I.

Setting: Initialize $x_{k,-1} = \psi_{k,-1} = 0$ and $w_{k,-1}$ arbitrary. For every agent k, repeat for $i = 0, 1, 2, \ldots$

$$\psi_{k,i} = w_{k,i-1} - \mu\nabla J_k(w_{k,i-1}) \tag{41a}$$

$$z_{k,i} = 2x_{k,i-1} - \sum_{s\in\mathcal{N}_k} a_{sk}(x_{s,i-1} - \psi_{s,i} + \psi_{s,i-1}) \tag{41b}$$

$$x_{k,i} = \sum_{s\in\mathcal{N}_k} a_{sk} z_{s,i} \tag{41c}$$

$$w_{k,i} = \mathbf{prox}_{\mu R}(x_{k,i}) \tag{41d}$$

Algorithm 3: Prox-ATC II.

Setting: Initialize $x_{k,-1} = \psi_{k,-1} = 0$ and $w_{k,-1}$ arbitrary. For every agent k, repeat for $i = 0, 1, 2, \ldots$

$$\psi_{k,i} = 2x_{k,i-1} - \mu\left(\nabla J_k(w_{k,i-1}) - \nabla J_k(w_{k,i-2})\right) \tag{42a}$$

$$z_{k,i} = \psi_{k,i} - \sum_{s\in\mathcal{N}_k} a_{sk}(x_{s,i-1} - w_{s,i-1} + w_{s,i-2}) \tag{42b}$$

$$x_{k,i} = \sum_{s\in\mathcal{N}_k} a_{sk} z_{s,i} \tag{42c}$$

$$w_{k,i} = \mathbf{prox}_{\mu R}(x_{k,i}) \tag{42d}$$

Rearranging the previous equation we get

$$w_i = \mathcal{A}(2w_{i-1} - \mathcal{A} w_{i-2} - \mu\mathcal{A}\left(\nabla\mathcal{J}(w_{i-1}) - \nabla\mathcal{J}(w_{i-2})\right))$$

which is recursion (13).

B. ATC-Tracking

In a similar manner we can show that (16) is equivalent to (15). From (16a) we have

$$\begin{aligned} &w_i - \mathcal{A} w_{i-1} \\ &= \mathcal{A}(w_{i-1} - \mathcal{A} w_{i-2} - \mu\left(x_{i-1} - \mathcal{A} x_{i-2}\right)) \\ &\overset{(16b)}{=} \mathcal{A}(w_{i-1} - \mathcal{A} w_{i-2} - \mu\left(\nabla\mathcal{J}(w_{i-1}) - \nabla\mathcal{J}(w_{i-2})\right)) \end{aligned}$$

Rearranging the previous equation we get

$$w_i = \mathcal{A}(2w_{i-1} - \mathcal{A} w_{i-2} - \mu\left(\nabla\mathcal{J}(w_{i-1}) - \nabla\mathcal{J}(w_{i-2})\right))$$

which is recursion (15).

REFERENCES

[1] J. Xu, S. Zhu, Y. C. Soh, and L. Xie, "Augmented distributed gradient methods for multi-agent optimization under uncoordinated constant stepsizes," in *Proc. 54th IEEE Conf. Decis. Control*, 2015, pp. 2055–2060.

[2] A. Nedić, A. Olshevsky, W. Shi, and C. A. Uribe, "Geometrically convergent distributed optimization with uncoordinated step-sizes," in *Proc. Amer. Control Conf.*, May 2017, pp. 3950–3955.

[3] P. Di Lorenzo and G. Scutari, "Next: In-network nonconvex optimization," *IEEE Trans. Signal Inf. Process. Over Netw.*, vol. 2, no. 2, pp. 120–136, Jun. 2016.

[4] G. Scutari and Y. Sun, "Distributed nonconvex constrained optimization over time-varying digraphs," *Math. Program.*, vol. 176, no. 1-2, pp. 497–544, 2019.

[5] Y. Sun, G. Scutari, and D. Palomar, "Distributed nonconvex multiagent optimization over time-varying networks," in *Proc. Asilomar Conf. Signals, Syst. Comput.*, Nov. 2016, pp. 788–794.

[6] G. Qu and N. Li, "Harnessing smoothness to accelerate distributed optimization," *IEEE Trans. Control Netw. Syst.*, vol. 5, no. 3, pp. 1245–1260, Sep. 2018.

[7] A. Nedic, A. Olshevsky, and W. Shi, "Achieving geometric convergence for distributed optimization over time-varying graphs," *SIAM J. Optim.*, vol. 27, no. 4, pp. 2597–2633, 2017.

[8] W. Shi, Q. Ling, G. Wu, and W. Yin, "Extra: An exact first-order algorithm for decentralized consensus optimization," *SIAM J. Optim.*, vol. 25, no. 2, pp. 944–966, 2015.

[9] Q. Ling, W. Shi, G. Wu, and A. Ribeiro, "DLM: Decentralized linearized alternating direction method of multipliers," *IEEE Trans. Signal Process.*, vol. 63, no. 15, pp. 4051–4064, Aug. 2015.

[10] K. Yuan, B. Ying, X. Zhao, and A. H. Sayed, "Exact diffusion for distributed optimization and learning-Part I: Algorithm development," *IEEE Trans. Signal Process.*, vol. 67, no. 3, pp. 708–723, Feb. 2019.

[11] Z. Li, W. Shi, and M. Yan, "A decentralized proximal-gradient method with network independent step-sizes and separated convergence rates," *IEEE Trans. Signal Process.*, vol. 67, no. 17, pp. 4494–4506, Sep. 2019.

[12] S. Pu, W. Shi, J. Xu, and A. Nedić, "A push-pull gradient method for distributed optimization in networks," in *Proc. IEEE Conf. Decis. Control*, Dec. 2018, pp. 3385–3390.

[13] R. Xin and U. A. Khan, "A linear algorithm for optimization over directed graphs with geometric convergence," *IEEE Control Syst. Lett.*, vol. 2, no. 3, pp. 315–320, Jul. 2018.

[14] Y. Sun, A. Daneshmand, and G. Scutari, "Convergence rate of distributed optimization algorithms based on gradient tracking," May 2019, *arXiv:1905.02637*.

[15] D. Jakovetic, "A unification and generalization of exact distributed first-order methods," *IEEE Trans. Signal Inf Process. over Netw.*, vol. 5, no. 1, pp. 31–46, Mar. 2019.

[16] A. Sundararajan, B. Van Scoy, and L. Lessard, "A canonical form for first-order distributed optimization algorithms," in *Proc. Amer. Control Conf.*, Jul. 2019, pp. 4075–4080.

[17] A. Sundararajan, B. Van Scoy, and L. Lessard, "Analysis and design of first-order distributed optimization algorithms over time-varying graphs," *IEEE Trans. Control Netw. Syst.*, to be published.

[18] F. S. Cattivelli and A. H. Sayed, "Diffusion LMS strategies for distributed estimation," *IEEE Trans. Signal Process.*, vol. 58, no. 3, pp. 1035–1048, Mar. 2010.

[19] A. H. Sayed, *Adaptation, Learning, and Optimization Over Neworks.* (Foundations and Trends in Machine Learning), Boston, MA, USA: Now, vol. 7, no. 4-5, 2014, pp. 311–801.

[20] S. A. Alghunaim, K. Yuan, and A. H. Sayed, "A linearly convergent proximal gradient algorithm for decentralized optimization," in *Proc. Advances Neural Inf. Process. Syst.*, Dec. 2019, pp. 2844–2854.

[21] W. Shi, Q. Ling, G. Wu, and W. Yin, "A proximal gradient algorithm for decentralized composite optimization," *IEEE Trans. Signal Process.*, vol. 63, no. 22, pp. 6013–6023, Nov. 2015.

[22] T.-H. Chang, M. Hong, and X. Wang, "Multi-agent distributed optimization via inexact consensus ADMM," *IEEE Trans. Signal Process.*, vol. 63, no. 2, pp. 482–497, Jan. 2015.

[23] N. S. Aybat, Z. Wang, T. Lin, and S. Ma, "Distributed linearized alternating direction method of multipliers for composite convex consensus optimization," *IEEE Trans. Autom. Control*, vol. 63, no. 1, pp. 5–20, Jan. 2018.

[24] P. Bianchi, W. Hachem, and F. Iutzeler, "A coordinate descent primal-dual algorithm and application to distributed asynchronous optimization," *IEEE Trans. Autom. Control*, vol. 61, no. 10, pp. 2947–2957, Oct. 2016.

[25] B. Woodworth and N. Srebro, "Tight complexity bounds for optimizing composite objectives," in *Proc. 30th Int. Conf. Neural Inf. Process. Syst.*, Dec. 2016, pp. 3639–3647.

[26] Y. Arjevani and O. Shamir, "Communication complexity of distributed convex learning and optimization," in *Proc. 28th Int. Con. Neural Inf. Process. Syst.*, Dec. 2015, pp. 1756–1764.

[27] C. A. Uribe, S. Lee, A. Gasnikov, and A. Nedić, "A dual approach for optimal algorithms in distributed optimization over networks," *Optim. Methods Softw.*, pp. 1–40, 2020.

[28] N. Loizou and P. Richtárik, "A new perspective on randomized gossip algorithms," in *Proc. IEEE Global Conf. Signal Inform. Process.*, Dec. 2016, pp. 440–444.

[29] S. A. Alghunaim, E. K. Ryu, K. Yuan, and A. H. Sayed, "Decentralized proximal gradient algorithms with linear convergence rates," Sep. 2019, *arXiv:1909.06479*.

[30] K. Yuan, B. Ying, X. Zhao, and A. H. Sayed, "Exact diffusion for distributed optimization and learning-Part II: Convergence analysis," *IEEE Trans. Signal Process.*, vol. 67, no. 3, pp. 724–739, Feb. 2019.

[31] Y. Nesterov, *Introductory Lectures on Convex Optimization: A Basic Course*. Berlin, Germany: Springer, 2013.

[32] P. Latafat, N. M. Freris, and P. Patrinos, "A new randomized block-coordinate primal-dual proximal algorithm for distributed optimization," *IEEE Trans. Autom. Control*, vol. 64, no. 10, pp. 4050–4065, Oct. 2019.